Bobby AI — Project and Operations Guide

This guide describes the repository as it exists today: its services, modules, dependencies, configuration, request paths, local setup, YouTube workflow, security boundaries, and test commands. The source folders are already separated by responsibility. Keep those stable import paths; do not move modules unless the imports, extension package paths, and tests are updated together.

1. What Bobby is

Bobby is a Windows-first, voice-first desktop assistant. The browser UI joins a LiveKit room, the Python LiveKit agent handles the conversation with Google's Gemini Realtime model, and the agent invokes browser, Windows, keyboard, filesystem, and optional MCP tools. A FastAPI service issues LiveKit room tokens and distributes agent/tool status to the dashboard. An unpacked Chrome extension gives the agent a user-authorized bridge to Chrome tabs.

The web experience is a single-screen command center. It does not provide a chat composer or send button. Microphone, camera, screen-share, and session controls are explicit user actions. Camera and screen tracks are opt-in.

2. System architecture

```
flowchart LR
    Web[React and Vite dashboard] --&gt;|POST /api/token| API[FastAPI backend]
    API --&gt;|LiveKit URL and room token| Web
    Web &lt;--&gt;|audio and optional media| LK[LiveKit]
    LK &lt;--&gt;|room dispatch and agent media| Agent[Python LiveKit agent]
    Agent &lt;--&gt;|voice conversation| Gemini[Gemini Realtime]
    Agent --&gt;|fixed allowlisted actions| Bridge[Loopback Chrome bridge]
    Bridge &lt;--&gt;|WebSocket 127.0.0.1:8765/browser| Extension[Chrome extension]
    Extension --&gt;|authorized HTTP(S) tabs| Chrome[Google Chrome]
    Agent --&gt; Win[Windows and keyboard tools]
    Agent --&gt; FS[Filesystem MCP over stdio]
    Agent --&gt; Desktop[Desktop MCP over stdio]
    Agent --&gt; Gmail[Gmail MCP over stdio]
    Agent --&gt; GitHub[GitHub MCP over HTTPS]
    Agent --&gt;|session, state, and tool events| API
    API --&gt;|GET /api/activity and /ws/activity| Web
```

A voice session, end to end

1 The dashboard calls `POST /api/token`. The backend validates its LiveKit settings, creates a unique room and identity, signs a one-hour room token, and dispatches the configured agent.

2 The browser joins the room. It publishes microphone media only when the user starts/enables the session; camera and screen tracks are separately opt-in.

3 LiveKit starts `apps/agent/agent.py`. `apps/agent/runtime.py` loads the root environment, constructs the Gemini Realtime session, sets agent instructions, and registers tools/toolsets.

4 An agent tool publishes lifecycle/status events to `POST /api/activity`. FastAPI broadcasts them over `/ws/activity` and provides recent events from `GET /api/activity`.

5 The React app maps real activity events to its tool panel and running-tool animation. The demo

hook is used only when the live activity feed is not connected; it does not call tools.

3. Repository map and module responsibilities

```
apps/
  agent/
    agent.py                          LiveKit CLI entrypoint
    runtime.py                        Agent, prompts, tool/MCP setup, state and lifecycle
    activity.py                       Bounded/redacted transcript and activity publishing
    config/app_config.py              Agent name, owner and Gemini model/voice constants
    bridges/chrome_bridge.py          Authenticated loopback extension WebSocket/RPC transport
    tools/browser.py                  Search, browser operations and YouTube play tool
    tools/youtube.py                   YouTube query normalization and recency search URL
    tools/keyboard.py                 Restricted key presses and plain-text entry
    tools/windows.py                  Allowlisted Windows shortcuts and app/window control
    mcp_clients/desktop_mcp.py         Desktop MCP startup, discovery and tool allowlist
    mcp_clients/gmail_mcp.py           Gmail MCP startup, restricted environment and allowlist
    mcp_clients/github_mcp.py          Read-only remote GitHub MCP client
  backend/main.py                     Health, LiveKit token, activity HTTP and WebSocket API
  chrome-extension/
    manifest.json                     MV3 permissions and extension entrypoints
    service_worker.js                 Bridge reconnect, tab permissions and action orchestration
    page_script.js                    Page snapshots, YouTube selection/player and page actions
    popup.html / popup.js / popup.css Extension connection and user permission controls
  web/
    index.html                        Vite HTML entry
    vite.config.ts                    React/Tailwind/Vite configuration and root env loading
    package.json / package-lock.json  Front-end dependencies and repeatable npm installation
    src/main.tsx                      React application bootstrap
    src/App.tsx                       Session, activity, state and component wiring
    src/hooks/useBobby.ts             LiveKit connection, media and session lifecycle
    src/hooks/useActivityFeed.ts      Activity history/WebSocket connection and error handling
    src/hooks/useDemoEvents.ts        Local-only interface demonstration events
    src/lib/api.ts                    Backend URLs, activity types/parsing and token API
    src/components/assistantState.ts  Shared UI state type
    src/components/LiveKitVoiceHandler.tsx Audio levels, state and transcript handling
    src/components/NeuralGlobe.tsx    Three.js/R3F neural globe and audio-reactive scene
    src/components/command-center/    Avatar, top bar, controls, tools and command-center layout
      AvatarPanel.tsx                 State, caption, status and particle avatar wrapper
      ParticleAvatar.tsx              Three.js particle portrait renderer and mode switching
      particleAvatarParticles.ts      Procedural CORE/FEMALE/MALE particle targets
      particleAvatar.shaders.ts        GPU vertex and fragment shaders
      CommandCenter.tsx               Main screen composition and session controls
      ToolPanel.tsx                   Tool/MCP status groups and live lifecycle rows
      ToolExecutionHud.tsx             Current tool animation driven by actual activity events
      toolActivity.ts                  Activity correlation and running-tool selection
      TopBar.tsx                       Agent status, language and drawer controls
      MediaPreview.tsx                 LiveKit camera/screen share picture-in-picture
    src/index.css                      Layout, responsive styling and motion effects
mcp_servers/filesystem/
  server.py                           Filesystem MCP stdio server and tool registration
  tools.py                            Filesystem operations
  security.py                         Scope, path, link and sensitive-content checks
  config.py                           Allowed filesystem roots and configuration
tests/
  test_activity_api.py                Activity metadata, transcript retention and CORS tests
  test_chrome_bridge.py               Chrome bridge timeout behavior
  test_filesystem.py                  Filesystem safeguards and MCP integration tests
  test_windows.py                     Windows tool behavior tests
  test_youtube_search.py               YouTube recency and query URL tests
docs/
  ARCHITECTURE.md                     Concise architectural reference
  SETUP.md                            Windows setup and optional MCP authorization
  BOBBY_AI_PROJECT_GUIDE.md            Detailed project guide (source for the PDF)
requirements.txt                      Python runtime/test dependency list
.env.example                          Safe root environment template
```

Service and module details

Agent runtime. `runtime.py` owns the LiveKit agent lifecycle, model/session construction, voice instructions, tool registration, optional MCP startup and shutdown, and status events. `config/app_config.py` currently selects model `gemini-3.1-flash-live-preview` and voice `Leda`. The standalone tools are kept in `tools/`; MCP launchers are kept in `mcp_clients/`; the bridge transport is isolated in `bridges/`. `activity.py` bounds/redacts transcript content before publishing it to the backend.

Backend. `backend/main.py` exposes `GET /api/health`, `POST /api/token`, `POST /api/activity`, `GET /api/activity`, and `WS /ws/activity`. It retains up to 100 non-transcript events in memory. Transcript events are streamed live but are not stored in activity history.

Chrome integration. `chrome_bridge.py` listens only on loopback and accepts only the expected extension origin and a fixed action allowlist. The MV3 service worker holds the bridge connection, checks user-granted host/tab access, and orchestrates tab changes. `page_script.js` is injected into the current authorized page to read/use that page. It does not use Chrome DevTools or remote debugging.

Filesystem MCP. Bobby's local stdio server performs operations only within configured roots. The security module rejects traversal/out-of-scope paths, sensitive locations/content, and unsafe links. The server translates individual operation failures into actionable tool errors.

Optional MCPs. Desktop and Gmail MCP servers are local stdio child processes with constrained environments and tool allowlists. GitHub MCP is remote HTTPS and configured read-only. Optional toolset startup failures should appear in agent logs/activity; the remaining agent can still start without an optional MCP.

Web client. `App.tsx` joins the session/activity hooks and passes real state to the command center. R3F/Three.js render the globe and particle avatar. `ToolPanel` and `ToolExecutionHud` use the same backend lifecycle events; UI animation is not evidence that an action succeeded. Tool success or failure must come from the tool result.

4. Technologies and project versions

Versions below are from the repository dependency manifests. Range prefixes in `package.json` allow compatible later versions when reinstalling; `package-lock.json` pins the resolved npm tree.

Area	Technologies
Agent	Python, LiveKit Agents `1.8.10`, Google Gemini Realtime plugin `1.8.10`, aiohttp
Backend	FastAPI, Pydantic, Uvicorn, LiveKit API
Browser	Chrome Manifest V3, `chrome.tabs`, `chrome.scripting`, optional host permissions, WebSocket
Web	React `19`, TypeScript, Vite `8`, Tailwind CSS `4` Vite plugin
3D and animation	Three.js, `@react-three/fiber`, `@react-three/drei`, Motion for React, CSS
Realtime media	LiveKit client/components, WebRTC audio/video/screen tracks
MCP	LiveKit MCP client, stdio filesystem/desktop/Gmail servers, remote GitHub MCP
Tests	pytest, TypeScript compiler (`npm run lint`), Vite production build

The current web manifest uses React 19, not React 18. It uses the `motion` package and imports `motion/react`; it does not depend on the separate `framer-motion` package.

5. YouTube: expected behavior and troubleshooting

Playback path

The voice model calls `browser_play_youtube_video(query)` for a request to play a YouTube video. The tool rejects an empty or overlong query, then `build_youtube_search_url()` creates a YouTube results URL. For recognized "latest/recent" phrases (including common Hindi/Urdu terms), it removes stale years/recency words from the query and asks YouTube to sort by upload date. Specific song titles stay intact.

The agent sends `youtube.search_play` through the local bridge with a 90-second bounded deadline (ordinary Chrome RPC actions retain a 30-second deadline). The extension:

1. Validates that the search URL points to YouTube results.
2. Opens a new tab; it does not repurpose the user's existing tab.
3. Waits for the results page and selects a visible, non-promoted regular `/watch` result.
4. Checks that the result has a valid 11-character video ID and canonicalizes its watch URL.
5. Opens that video with an autoplay request and waits for its page/player.
6. Tries the player and visible YouTube play control, then verifies whether the player is actually running. It may click only a visible YouTube "Skip ad" control.
7. Returns selected title, URL, playback state, ad state and any browser playback error.

The agent must not say that playback started unless the returned playback state says it started. If `play()` raises `NotAllowedError`, Chrome's autoplay policy is the blocker, not the extension's tab permission. Bobby should leave the selected video open and ask the user to click YouTube's Play control once. No extension can safely or reliably bypass a browser's user-activation policy. An ad may play before the requested video; the agent reports that honestly and does not block or bypass ads.

Reliable test prompts

- "Search YouTube for `Hasbi Rabbi Jallallah Naat` but don't play it."
- "Play `Hasbi Rabbi Jallallah Naat` on YouTube and tell me the selected title."
- "Play the latest Naat on YouTube."
- "Play a video from this YouTube URL: [URL]." (The current dedicated tool is search-first; do not assume URL-based direct playback is supported.)
- "If Chrome blocks autoplay, leave the video open and tell me exactly what I need to click."

Troubleshooting checklist

1. Start the agent and a connected Bobby voice session. The extension bridge runs only while the agent session is active.
2. Open the Bobby extension popup and confirm it says connected.
3. Grant the extension all-sites access if it needs to inject into YouTube. Tab authorization and host-site permission are separate controls.
4. Reload the unpacked extension from `chrome://extensions` after changing extension files.

5 Confirm Chrome can open and play the selected URL normally. Account restrictions, age/region limits, private/deleted videos, connectivity, ads, and YouTube UI changes can prevent playback.

6 Read the actual tool response. “Chrome extension is not connected,” “no playable result,” “navigation timed out,” and “NotAllowedError” identify different failure points.

7 For `NotAllowedError`, focus the YouTube tab and click its Play button once. Extension permission is not equivalent to autoplay permission.

8 For a wrong selection, ask for a distinctive title/artist, confirm the returned title, and inspect whether the top result is actually the requested video. Search ranking is owned by YouTube and cannot guarantee the newest or exact match.

The extension's page selectors depend on YouTube's current DOM; YouTube can change that DOM at any time. Keep selectors narrow, validate the ID/title/host before navigation, and test against current Chrome/YouTube instead of weakening URL checks.

6. Configuration and secrets

Copy `.env.example` to `.env` at the repository root. Never commit the local `.env`, OAuth files, or personal access tokens. The web Vite config reads the root environment directory. Use `VITE_API_URL` only when the backend is not at its default local URL.

Setting	Purpose
`GOOGLE_API_KEY`	Gemini Realtime credentials for the agent
`LIVEKIT_URL`, `LIVEKIT_API_KEY`, `LIVEKIT_API_SECRET`	Room URL and backend token signing
`AGENT_NAME`	LiveKit dispatch agent name; keep it consistent between backend and agent
`BACKEND_URL`	Agent activity publish target
`INTERNAL_KEY`	Optional shared credential protecting activity POSTs
`CORS_ORIGINS`	Allowed web origins; defaults cover localhost/127.0.0.1 on 3000/5173
`VITE_API_URL`	Backend base URL for the dashboard
`BOBBY_TOOLSETS`	Optional MCP list: `desktop`, `gmail`, `github`; empty disables optional MCPs
`GITHUB_PERSONAL_ACCESS_TOKEN`	GitHub MCP token; use minimal read-only repository scope
`BOBBY_FILESYSTEM_ACCESS`, `BOBBY_FILESYSTEM_ROOTS`	Filesystem authorization scope
`DESKTOP_TOUCH_MCP_FETCH_TIMEOUT_MS`	Optional desktop package download timeout
`DISABLE_HMR`	Optional Vite file-watch/HMR switch

Gmail OAuth client keys and generated credentials live under `%USERPROFILE%\.gmail-mcp`, outside the repository. The Gmail MCP is enabled only after its OAuth setup is complete.

7. Start the project on Windows

Prerequisites: Windows 10/11, Python compatible with the Python dependencies, Node/npm compatible with the locked Vite version, Google Chrome, a LiveKit project, and a Google AI API key. Configure the optional Gmail/GitHub credentials only when enabling those toolsets.

```
Set-Location D:\bobby
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Copy-Item .env.example .env

Set-Location D:\bobby\apps\web
npm ci
```

Load the unpacked extension from `D:\bobby\apps\chrome-extension` at `chrome://extensions`. Then run each process in its own PowerShell window:

```
# Backend
Set-Location D:\bobby\apps\backend
..\..\.venv\Scripts\python.exe -m uvicorn main:app --host 127.0.0.1 --port 8000 --reload

# Voice agent
Set-Location D:\bobby
.\.venv\Scripts\python.exe .\apps\agent\agent.py dev

# Web UI
Set-Location D:\bobby\apps\web
npm run dev
```

Open `http://localhost:3000`. Check `http://127.0.0.1:8000/api/health` for backend status. `http://localhost:3000/?demo=1` demonstrates the interface without joining LiveKit or executing tools.

8. Verification and maintenance

From `D:\bobby`:

```
.\.venv\Scripts\python.exe -m pytest
.\.venv\Scripts\python.exe -m compileall apps mcp_servers
```

From `D:\bobby\apps\web`:

```
npm run lint
npm run build
```

Extension syntax smoke checks:

```
Set-Location D:\bobby
node --check apps\chrome-extension\service_worker.js
node --check apps\chrome-extension\page_script.js
node --check apps\chrome-extension\popup.js
```

When changing a tool, test its output and failure path as well as its activity event. When changing the extension, reload it in Chrome and test against a real authorized tab. Keep browser permissions and MCP tool allowlists explicit. Do not treat an animated “running” indicator as proof of successful execution.

9. Boundaries and known limitations

- Browser control requires a running agent/bridge, a connected extension, user-authorized tabs, and

host access. Chrome internal pages and non-HTTP(S) pages cannot be automated.

- Automated YouTube playback is best-effort. The browser may require a trusted user click, and

content, account, network, regional, or ad conditions can block the requested video.

- Search results are ranked by YouTube. Sorting by upload date does not verify that a result is the newest eligible video or the exact work intended; the returned title is the evidence.
- The backend activity history is in-memory and limited to 100 non-transcript events. It is not a durable audit log.
- Demo activity is mock UI data and never invokes the agent tools.
- Desktop automation may not control elevated/admin windows because of Windows integrity/UIPI boundaries. Bobby must not elevate itself or bypass that boundary.
- A voice session requires functioning LiveKit and Gemini configuration. Optional MCP availability depends on package downloads, permissions, OAuth, and network connectivity.
- Project license is not specified; add a license before redistributing.